

The toral–unipotent factorization $G(\mathfrak{m}) = HU$: a machine-checked proof

Abstract

We present a formal verification of Theorem 3.6 of the source (thm: `gsimple g m eq h u`). The theorem concerns the group $G(\mathfrak{m})$ given by generators and relations attached to the coefficients of the modular function J . It states that $G(\mathfrak{m}) = HU$, where H is the toral subgroup and U is the unipotent subgroup. The formalization takes the coefficient function as an arbitrary parameter $c: \mathbb{N} \rightarrow \mathbb{N}$ and needs no hypothesis on it. The formal proof follows the source argument: it shows that the set HU is a subgroup containing every generator. Closure under products and inverses rests on the previously verified fact that H normalizes U (Theorem 3.4 and Lemma 3.5 of the source).

Source	Carbone et al., G-Simple (draft), <code>GSimple_Theorem_3_6</code>
Library	<code>libraries/gsimple.lean</code> , 60 declarations
Lean	<code>leanprover/lean4:v4.23.0</code> , Mathlib 37df177aaa
Machine checks	compiles; no sorry; statement matches target; module build
Axioms	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Run	<code>srv_20261006_173044_GSimple_Theorem_3_6</code>

Contents

1	Statement	1
2	Formalization	3
3	Proof	9
3.1	Overview	9
3.2	Proof of the theorem	12
4	Independent review	13
A	Complete Lean file	14

1 Statement

Let $c(j)$ denote the coefficient of q^j in the modular function

$$J(q) = \sum_{j \geq -1} c(j)q^j = \frac{1}{q} + 196884q + 21493760q^2 + 864299970q^3 + \cdots,$$

so that $c(-1) = 1$, $c(0) = 0$, $c(1) = 196884$, and so on.

Define the index set

$$E = \{(\ell, j, k) \mid j \in \mathbb{N}, 1 \leq k \leq c(j), 0 \leq \ell < j\}$$

and, for $0 \leq \ell < j$, the number

$$c_{\ell j} = (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell).$$

Let $\mathcal{X}$ be the set of symbols

$$H_1(s), H_2(s), X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u) \quad (s \in \mathbb{C}^\times, u \in \mathbb{C}, (\ell, j, k) \in E).$$

Let $F(\mathcal{X})$ be the free group on $\mathcal{X}$. The symbols $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$ are called the *unipotent generators*. The commutator is $(a, b) = aba^{-1}b^{-1}$.

For $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$ and $(\ell, j, k), (m, p, q) \in E$, put

$$\tilde{w}_{-1}(s) := X_{-1}(s) Y_{-1}(-s^{-1}) X_{-1}(s), \quad \tilde{w}_{-1} := \tilde{w}_{-1}(1), \quad (\text{Re:0})$$

$$\tilde{w}_{\ell,jk}(s) := X_{\ell,jk}(s) Y_{\ell,jk}\left(\frac{-s^{-1}}{c_{\ell j}}\right) X_{\ell,jk}(s), \quad \tilde{w}_{\ell,jk} := \tilde{w}_{\ell,jk}(1). \quad (\text{Im:0})$$

The set $\mathcal{R} \subseteq F(\mathcal{X})$ consists of the following relations.

$$H_1(s)H_1(t) = H_1(st), \quad (\text{H:1a})$$

$$H_2(s)H_2(t) = H_2(st), \quad (\text{H:1b})$$

$$H_1(s)H_2(t) = H_2(t)H_1(s), \quad (\text{H:2})$$

$$X_{-1}(u)X_{-1}(v) = X_{-1}(u+v), \quad (\text{Re:1a})$$

$$Y_{-1}(u)Y_{-1}(v) = Y_{-1}(u+v), \quad (\text{Re:1b})$$

$$Y_{-1}(-t)X_{-1}(s)Y_{-1}(t) = X_{-1}(-t^{-1})Y_{-1}(-t^2s)X_{-1}(t^{-1}), \quad (\text{Re:2})$$

$$\tilde{w}_{-1}(s)\tilde{w}_{-1} = H_1(-s)H_2(-s^{-1}), \quad (\text{Re:3})$$

$$\tilde{w}_{-1}X_{-1}(u)\tilde{w}_{-1}^{-1} = Y_{-1}(-u), \quad (\text{Re:4a})$$

$$\tilde{w}_{-1}Y_{-1}(u)\tilde{w}_{-1}^{-1} = X_{-1}(-u), \quad (\text{Re:4b})$$

$$\tilde{w}_{-1}H_1(s)\tilde{w}_{-1}^{-1} = H_2(s), \quad (\text{Re:5a})$$

$$\tilde{w}_{-1}H_2(s)\tilde{w}_{-1}^{-1} = H_1(s), \quad (\text{Re:5b})$$

$$H_1(s)X_{-1}(u)H_1(s)^{-1} = X_{-1}(su), \quad (\text{Re:6a})$$

$$H_2(s)X_{-1}(u)H_2(s)^{-1} = X_{-1}(s^{-1}u), \quad (\text{Re:6b})$$

$$H_1(s)Y_{-1}(u)H_1(s)^{-1} = Y_{-1}(s^{-1}u), \quad (\text{Re:6c})$$

$$H_2(s)Y_{-1}(u)H_2(s)^{-1} = Y_{-1}(su), \quad (\text{Re:6d})$$

$$X_{\ell,jk}(u)X_{\ell,jk}(v) = X_{\ell,jk}(u+v), \quad (\text{Im:1a})$$

$$Y_{\ell,jk}(u)Y_{\ell,jk}(v) = Y_{\ell,jk}(u+v), \quad (\text{Im:1b})$$

$$Y_{\ell,jk}(-t)X_{\ell,jk}(s)Y_{\ell,jk}(t) = X_{\ell,jk}\left(\frac{-t^{-1}}{c_{\ell j}}\right)Y_{\ell,jk}(-c_{\ell j}t^2s)X_{\ell,jk}\left(\frac{t^{-1}}{c_{\ell j}}\right), \quad (\text{Im:2})$$

$$\tilde{w}_{\ell,jk}(s^{(\ell+1)(j-\ell)})\tilde{w}_{\ell,jk} = H_1((-s)^{j-\ell})H_2((-s)^{\ell+1}), \quad (\text{Im:3})$$

$$\tilde{w}_{\ell,jk} X_{\ell,jk}(u) \tilde{w}_{\ell,jk}^{-1} = Y_{\ell,jk}(-u/c_{\ell j}), \quad (\text{Im:4a})$$

$$\tilde{w}_{\ell,jk} Y_{\ell,jk}(u) \tilde{w}_{\ell,jk}^{-1} = X_{\ell,jk}(-c_{\ell j} u), \quad (\text{Im:4b})$$

$$\tilde{w}_{\ell,jk} H_1(s^{j-\ell}) \tilde{w}_{\ell,jk}^{-1} = H_2(s^{-(\ell+1)}), \quad (\text{Im:5a})$$

$$\tilde{w}_{\ell,jk} H_2(s^{\ell+1}) \tilde{w}_{\ell,jk}^{-1} = H_1(s^{-(j-\ell)}), \quad (\text{Im:5b})$$

$$H_1(s) X_{\ell,jk}(u) H_1(s)^{-1} = X_{\ell,jk}(s^{\ell+1} u), \quad (\text{Im:6a})$$

$$H_2(s) X_{\ell,jk}(u) H_2(s)^{-1} = X_{\ell,jk}(s^{j-\ell} u), \quad (\text{Im:6b})$$

$$H_1(s) Y_{\ell,jk}(u) H_1(s)^{-1} = Y_{\ell,jk}(s^{-(\ell+1)} u), \quad (\text{Im:6c})$$

$$H_2(s) Y_{\ell,jk}(u) H_2(s)^{-1} = Y_{\ell,jk}(s^{-(j-\ell)} u), \quad (\text{Im:6d})$$

$$(X_{-1}(u), X_{j-1,jk}(v)) = 1, \quad (\text{U:1a})$$

$$(Y_{-1}(u), Y_{j-1,jk}(v)) = 1, \quad (\text{U:1b})$$

$$(Y_{-1}(u), X_{0,jk}(v)) = 1, \quad (\text{U:1c})$$

$$(X_{-1}(u), Y_{0,jk}(v)) = 1, \quad (\text{U:1d})$$

$$(X_{\ell,jk}(u), Y_{m,pq}(v)) = 1 \quad \text{if } j \neq p \text{ or } k \neq q \text{ or } |\ell - m| > 1, \quad (\text{U:2})$$

$$(X_{1,2k}(u), Y_{0,2k}(v)) = X_{-1}(uv), \quad (\text{U:3a})$$

$$(X_{0,2k}(u), Y_{1,2k}(v)) = Y_{-1}(uv), \quad (\text{U:3b})$$

$$\tilde{w}_{-1} X_{\ell,jk}(u) \tilde{w}_{-1}^{-1} = X_{j-1-\ell,jk}((-1)^{j-\ell-1} u), \quad (\text{U:4a})$$

$$\tilde{w}_{-1} Y_{\ell,jk}(u) \tilde{w}_{-1}^{-1} = Y_{j-1-\ell,jk}((-1)^{j-\ell-1} u). \quad (\text{U:4b})$$

The group is

$$G(\mathfrak{m}) = \langle \mathcal{X} \mid \mathcal{R} \rangle = F(\mathcal{X})/N_{\mathcal{R}},$$

where $N_{\mathcal{R}}$ is the normal closure of $\mathcal{R}$. The *toral subgroup* H is the subgroup of $G(\mathfrak{m})$ generated by $\{H_1(s), H_2(s) \mid s \in \mathbb{C}^\times\}$. The *unipotent subgroup* U is the subgroup generated by all unipotent generators:

$$U = \langle X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u) \mid u \in \mathbb{C}, (\ell, j, k) \in E \rangle.$$

Theorem 1.1 (Theorem 3.6 of the source). $G(\mathfrak{m}) = HU$.

2 Formalization

The coefficient function. The formalization does not define the coefficients of J . It takes an arbitrary function $c: \mathbb{N} \rightarrow \mathbb{N}$ as a parameter, as the problem set allows. All objects below depend on c . The source's group is obtained by taking $c(j)$ to be the q^j -coefficient of J for $j \in \mathbb{N}$; these are non-negative integers, with $c(0) = 0$. The coefficient $c(-1)$ plays no role, since E involves only $j \in \mathbb{N}$.

The present theorem needs no hypothesis on c . In particular, the hypothesis $c(2) \geq 1$ used elsewhere in the library is not needed here.

The library. The following definitions and results come from earlier verified work. The definitions are stated here in mathematical terms.

Definition 2.1 (Index set). For $\ell, j, k \in \mathbb{N}$, the predicate $\text{Valid}_c(\ell, j, k)$ means

$$\ell < j \quad \text{and} \quad 1 \leq k \leq c(j),$$

that is, $(\ell, j, k) \in E$.

Lean: Valid — library, verified earlier

```

| /-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
| def Valid (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) : Prop := 1 < j  $\wedge$  1  $\leq$  k  $\wedge$  k  $\leq$  c j

```

Definition 2.2 (Generating symbols). The type Gen_c of symbols has six kinds of elements:

- $H_1(s)$ and $H_2(s)$ for $s \in \mathbb{C}^\times$;
- $X_{-1}(u)$ and $Y_{-1}(u)$ for $u \in \mathbb{C}$;
- $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$ for $\ell, j, k \in \mathbb{N}$, a proof of $\text{Valid}_c(\ell, j, k)$, and $u \in \mathbb{C}$.

Proofs of a proposition are all equal in Lean, so this is the set $\mathcal{X}$ of the source.

The free group $F(\mathcal{X})$ is F_c . The elements $h_1(s)$, $h_2(s)$, $x_{-1}(u)$, $y_{-1}(u)$, $x_{\ell,jk}(u)$, $y_{\ell,jk}(u)$ of F_c are the corresponding basis elements.

Lean: Gen — library, verified earlier

```

| /-- The generating symbols. -/
| inductive Gen (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Type
| H1 :  $\mathbb{C}^\times \rightarrow \text{Gen } c$ 
| H2 :  $\mathbb{C}^\times \rightarrow \text{Gen } c$ 
| Xm :  $\mathbb{C} \rightarrow \text{Gen } c$ 
| Ym :  $\mathbb{C} \rightarrow \text{Gen } c$ 
| X (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c
| Y (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c

```

Lean: F — library, verified earlier

```

| abbrev F (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) := FreeGroup (Gen c)

```

Lean: h1 — library, verified earlier

```

| def h1 {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c := FreeGroup.of (Gen.H1 s)

```

Lean: h2 — library, verified earlier

```

| def h2 {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c := FreeGroup.of (Gen.H2 s)

```

Lean: xm — library, verified earlier

```

| def xm {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (u :  $\mathbb{C}$ ) : F c := FreeGroup.of (Gen.Xm u)

```

Lean: ym — library, verified earlier

```

| def ym {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (u :  $\mathbb{C}$ ) : F c := FreeGroup.of (Gen.Ym u)

```

Lean: xx — library, verified earlier

```
def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
```

Lean: yy — library, verified earlier

```
def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)
```

Definition 2.3 (Auxiliary elements). The library defines the following elements.

- The complex number

$$c_{\ell j} = (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j \dot{-} \ell),$$

where $\dot{-}$ is truncated subtraction on $\mathbb{N}$. It equals $j - \ell$ whenever $\ell \leq j$.

- The commutator $\text{cm}(a, b) = aba^{-1}b^{-1}$.
- The relator $\text{eqr}(a, b) = ab^{-1}$, which encodes the equation $a = b$.
- The words $\tilde{w}_{-1}(s)$ and $\tilde{w}_{\ell, jk}(s)$ for $s \in \mathbb{C}^\times$, defined exactly as in (Re:0) and (Im:0).

Lean: cc — library, verified earlier

```
/-- c_{\ell j} = (-1)^{\ell+1} * binom(j-1, \ell) * (\ell+1) * (j-\ell). -/
noncomputable def cc (l j : ℕ) : ℂ :=
  (-1 : ℂ) ^ (l + 1) * ((j - 1).choose l : ℕ) : ℂ * ((l : ℂ) + 1) * ((j - l : ℕ) : ℂ)
```

Lean: cm — library, verified earlier

```
/-- commutator (a,b) = a b a^{-1} b^{-1} -/
def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a^{-1} * b^{-1}
```

Lean: eqr — library, verified earlier

```
/-- relator for the equation a = b -/
def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b^{-1}
```

Lean: wm — library, verified earlier

```
noncomputable def wm {c : ℕ → ℕ} (s : ℂ^\times) : F c :=
  xm (s : ℂ) * ym (-(s : ℂ)^{-1}) * xm (s : ℂ)
```

Lean: ww — library, verified earlier

```
noncomputable def ww {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
  xx l j k h (s : ℂ) * yy l j k h (-(s : ℂ)-1) / cc l j) * xx l j k h (s : ℂ)
```

Definition 2.4 (Relators and the group). The predicate Rel_c on F_c has one constructor for each relation of the source. The relator is ab^{-1} for a relation $a = b$, and (a, b) itself for a relation $(a, b) = 1$. Its parameters range over the same sets as in the source: $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$, and triples in E . A few points of the encoding should be noted.

- In (Im:5a,b) and (Im:6c,d), the negative powers s^{-n} are written $(s^n)^{-1}$.
- (U:1a,b) are indexed by j, k with $(j-1, j, k) \in E$. (U:1c,d) are indexed by j, k with $(0, j, k) \in E$. (U:3a,b) are indexed by k with $(0, 2, k), (1, 2, k) \in E$.
- (U:4a,b) carry the additional hypothesis $(j-1-\ell, j, k) \in E$. This follows from $(\ell, j, k) \in E$, so the relations imposed are exactly those of the source. All subtractions there are truncated, but they are exact because $\ell < j$.

The set Rels_c collects these relators. The group $G_c = \text{Grp}_c$ is the presented group F_c/N , where N is the normal closure of Rels_c .

We write $\pi : F_c \rightarrow G_c$ for the quotient map, and use the same letters for generators and their images in G_c .

Lean: Rel — library, verified earlier

```
/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂ×) : Rel c (eqv (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂ×) : Rel c (eqv (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂ×) : Rel c (eqv (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqv (xm u * xm v) (xm (u + v)))
| Re1b (u v : ℂ) : Rel c (eqv (ym u * ym v) (ym (u + v)))
| Re2 (s t : ℂ×) : Rel c (eqv (ym (-(t : ℂ)) * xm (s : ℂ) * ym (t : ℂ))
  (xm (-(t : ℂ)-1) * ym (-(t : ℂ) ^ 2 * (s : ℂ)) * xm ((t : ℂ)-1)))
| Re3 (s : ℂ×) : Rel c (eqv (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqv (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u : ℂ) : Rel c (eqv (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s : ℂ×) : Rel c (eqv (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s : ℂ×) : Rel c (eqv (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s : ℂ×) (u : ℂ) : Rel c (eqv (h1 s * xm u * (h1 s)-1) (xm ((s : ℂ) * u)))
| Re6b (s : ℂ×) (u : ℂ) : Rel c (eqv (h2 s * xm u * (h2 s)-1) (xm ((s : ℂ)-1 * u)))
| Re6c (s : ℂ×) (u : ℂ) : Rel c (eqv (h1 s * ym u * (h1 s)-1) (ym ((s : ℂ)-1 * u)))
| Re6d (s : ℂ×) (u : ℂ) : Rel c (eqv (h2 s * ym u * (h2 s)-1) (ym ((s : ℂ) * u)))
| Im1a (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqv (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqv (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))
| Im2 (l j k : ℕ) (h : Valid c l j k) (s t : ℂ×) :
  Rel c (eqv (yy l j k h (-(t : ℂ)) * xx l j k h (s : ℂ) * yy l j k h (t : ℂ))
    (xx l j k h (-(t : ℂ)-1) / cc l j) *
    yy l j k h (-(cc l j) * (t : ℂ) ^ 2 * (s : ℂ)) *
    xx l j k h ((t : ℂ)-1 / cc l j)))
| Im3 (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
```

```

    Rel c (eqr (ww 1 j k h (s ^ ((1 + 1) * (j - 1))) * ww 1 j k h 1)
      (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (1 + 1))))
| Im4a (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
    Rel c (eqr (ww 1 j k h 1 * xx 1 j k h u * (ww 1 j k h 1)-1) (yy 1 j k h (-u / cc 1 j)))
| Im4b (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
    Rel c (eqr (ww 1 j k h 1 * yy 1 j k h u * (ww 1 j k h 1)-1) (xx 1 j k h (-cc 1 j * u)))
| Im5a (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
    Rel c (eqr (ww 1 j k h 1 * h1 (s ^ (j - 1)) * (ww 1 j k h 1)-1) (h2 ((s ^ (1 + 1))-1)))
| Im5b (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
    Rel c (eqr (ww 1 j k h 1 * h2 (s ^ (1 + 1)) * (ww 1 j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
    Rel c (eqr (h1 s * xx 1 j k h u * (h1 s)-1) (xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)))
| Im6b (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
    Rel c (eqr (h2 s * xx 1 j k h u * (h2 s)-1) (xx 1 j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
    Rel c (eqr (h1 s * yy 1 j k h u * (h1 s)-1) (yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)))
| Im6d (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
    Rel c (eqr (h2 s * yy 1 j k h u * (h2 s)-1) (yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
    Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
    Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
    Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
    Rel c (cm (xm u) (yy 0 j k h v))
| U2 (l j k : ℕ) (h : Valid c l j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ)
    (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((l : ℤ) - (m : ℤ))) :
    Rel c (cm (xx 1 j k h u) (yy m p q h' v))
| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
    Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
    Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
    Rel c (eqr (wm 1 * xx 1 j k h u * (wm 1)-1)
      (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
    Rel c (eqr (wm 1 * yy 1 j k h u * (wm 1)-1)
      (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

```

Lean: Rels — library, verified earlier

```
def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}
```

Lean: Grp — library, verified earlier

```

/-- The group  $G(m) = \langle X \mid R \rangle$ . -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

```

Definition 2.5 (Toral and unipotent subgroups). The set $\text{torusGens}_c \subseteq G_c$ consists of the elements $\pi(h_1(s))$ and $\pi(h_2(s))$ for $s \in \mathbb{C}^\times$. The subgroup $H = \text{torusSub}_c$ is the subgroup it generates.

The set unipGens_c consists of the images of

$$x_{-1}(u), \quad y_{-1}(u), \quad x_{\ell,jk}(u), \quad y_{\ell,jk}(u) \quad (u \in \mathbb{C}, (\ell, j, k) \in E).$$

The subgroup $U = \text{unipSub}_c$ is the subgroup it generates.

These are precisely the subgroups H and U of the source.

Lean: torusGens — library, verified earlier

```

/-- Generators of the toral subgroup: the images of H_1(s) and H_2(s) in G(m). -/
def torusGens (c : ℕ → ℕ) : Set (Grp c) :=
  {g | ∃ s : ℂ×, g = PresentedGroup.mk (Rels c) (h1 s) ∨
    g = PresentedGroup.mk (Rels c) (h2 s)}

```

Lean: torusSub — library, verified earlier

```

/-- The toral subgroup H of G(m). -/
def torusSub (c : ℕ → ℕ) : Subgroup (Grp c) := Subgroup.closure (torusGens c)

```

Lean: unipGens — library, verified earlier

```

/-- The unipotent generators X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u), viewed in G(m). -/
def unipGens (c : ℕ → ℕ) : Set (Grp c) :=
  {g | (∃ u : ℂ, g = PresentedGroup.mk (Rels c) (xm u)) ∨
    (∃ u : ℂ, g = PresentedGroup.mk (Rels c) (ym u)) ∨
    (∃ (l j k : ℕ) (h : Valid c l j k) (u : ℂ),
      g = PresentedGroup.mk (Rels c) (xx l j k h u)) ∨
    (∃ (l j k : ℕ) (h : Valid c l j k) (u : ℂ),
      g = PresentedGroup.mk (Rels c) (yy l j k h u))}

```

Lean: unipSub — library, verified earlier

```

/-- The unipotent subgroup U of G(m). -/
def unipSub (c : ℕ → ℕ) : Subgroup (Grp c) := Subgroup.closure (unipGens c)

```

Library results used in this proof. The library contains many verified results about G_c , for instance the perfectness of G_c (Theorem 3.3 of the source, `gsimple_g_m_perfect`). Only the following are used here.

`subgroup_top_of_gens`: for every c , every subgroup $N \leq G_c$ that contains the image of every symbol in Gen_c equals G_c . This holds because the images of the symbols generate the presented group.

Lean: `subgroup_top_of_gens` — library, verified earlier

```

theorem subgroup_top_of_gens (c : ℕ → ℕ) (N : Subgroup (GM.Grp c))
  (h : ∀ g : GM.Gen c, (PresentedGroup.of g : GM.Grp c) ∈ N) : N = ⊤ := by
  have hgen : Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c → GM.Grp c) ⊆
    → N.carrier := by
    rintro _ ⟨g, rfl⟩
    exact h g
  have hclosure : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) :
    → GM.Gen c → GM.Grp c)) ≤ N :=
    ((Subgroup.closure_le (K := N)).mpr hgen)
  have htop : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c
    → → GM.Grp c)) = ⊤ :=
    PresentedGroup.closure_range_of (GM.Rels c)
  rw [htop] at hclosure
  exact top_le_iff.mp hclosure

```

`gsimple_torus_normalizes_unipotent` (Theorem 3.4 of the source): for every c , every $h \in H$ and every $u \in U$, we have $huh^{-1} \in U$.

Lean: gsimple_torus_normalizes_unipotent — library, verified earlier

```

theorem gsimple_torus_normalizes_unipotent (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c, h * u * h-1 ∈ GM.unipSub c := by
  intro h hh u hu
  exact closure_normalizes (GM.torusGens c) (GM.unipGens c) (torusGens_inv c)
  (fun t ht => conj_closure_of_gens (GM.unipGens c) t
    (fun g hg => Subgroup.subset_closure (torusGens_conj c t ht g hg))) h hh u hu

-- From run srv_20261006_173043_GSimple_Lemma_3_5 (GSimple_Lemma_3_5), proved.

```

gsimple_torus_unipotent_commute (Lemma 3.5 of the source): for every c , every $h \in H$ and every $u \in U$, there is $u' \in U$ with $uh = hu'$. It is proved with $u' = h^{-1}uh$, using the previous result.

Lean: gsimple_torus_unipotent_commute — library, verified earlier

```

theorem gsimple_torus_unipotent_commute (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c,
  ∃ u' ∈ GM.unipSub c, u * h = h * u' := by
  intro h hh u hu
  refine ⟨h-1 * u * h-1-1,
    gsimple_torus_normalizes_unipotent c h-1 (Subgroup.inv_mem _ hh) u hu, ?_⟩
  exact conj_inv_rewrite h u

```

The target statement. Lean: target statement gsimple_g_m_eq_h_u, fixed before the proof; solution is checked to have exactly this type

```

theorem gsimple_g_m_eq_h_u (c : ℕ → ℕ) :
  ∀ g : GM.Grp c, ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, g = h * u

```

The target statement, proved below as solution, reads as follows. For every $c: \mathbb{N} \rightarrow \mathbb{N}$ and every $g \in G_c$, there exist $h \in H$ and $u \in U$ with $g = hu$.

This is the inclusion $G_c \subseteq HU$. The reverse inclusion $HU \subseteq G_c$ holds trivially in the formalization, because products of elements of G_c lie in G_c by typing. The source's equality $G(\mathfrak{m}) = HU$ is therefore exactly what is proved.

The function c stands for the coefficients of J . Since no hypothesis on c is needed, the theorem holds in particular for the source's choice of c .

3 Proof

3.1 Overview

Write $HU = \{hu \mid h \in H, u \in U\} \subseteq G_c$. The source proof has three steps, and the formal proof follows them:

1. HU is closed under multiplication (Lemma 3.1), by Lemma 3.5 of the source.
2. HU is closed under inversion (Lemma 3.2), again by Lemma 3.5.
3. HU contains every generator (Lemma 3.6).

The formal proof makes one point explicit that the source leaves implicit: $1 \in HU$ (Lemma 3.3). This is needed to obtain a subgroup.

The last step uses the library result `subgroup_top_of_gens`: a subgroup containing all generators is the whole group. Lemma 3.5 is the library result `gsimple_torus_unipotent_commute`.

Lemmas 3.4 and 3.5 record the inclusions $H \subseteq HU$ and $U \subseteq HU$. They were proved as auxiliaries but are not used in the final argument: Lemma 3.6 handles the generators directly.

There is no mathematical deviation from the source proof.

Lemma 3.1 (`hu_mul`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$ and $a, b \in G_c$. Suppose $a = h_1 u_1$ and $b = h_2 u_2$ with $h_1, h_2 \in H$ and $u_1, u_2 \in U$. Then there exist $h \in H$ and $u \in U$ with $ab = hu$.*

Lean: `hu_mul` — proved by AIProver, job 5, session 1

```

theorem hu_mul (c : ℕ → ℕ) (a b : GM.Grp c)
  (ha : ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a = h * u)
  (hb : ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, b = h * u) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a * b = h * u := by
  rcases ha with ⟨h₁, hh₁, u₁, hu₁, ha_eq⟩
  rcases hb with ⟨h₂, hh₂, u₂, hu₂, hb_eq⟩
  rcases gsimple_torus_unipotent_commute c h₂ hh₂ u₁ hu₁ with ⟨u₁', hu₁', hu_comm⟩
  refine ⟨h₁ * h₂, Subgroup.mul_mem _ hh₁ hh₂, u₁' * u₂, Subgroup.mul_mem _ hu₁' hu₂, ?_⟩
  calc
    a * b = (h₁ * u₁) * (h₂ * u₂) := by rw [ha_eq, hb_eq]
    _ = h₁ * (u₁ * h₂) * u₂ := by group
    _ = h₁ * (h₂ * u₁') * u₂ := by rw [hu_comm]
    _ = (h₁ * h₂) * (u₁' * u₂) := by group

```

Proof. Apply `gsimple_torus_unipotent_commute` to $h_2 \in H$ and $u_1 \in U$. This gives $u'_1 \in U$ with $u_1 h_2 = h_2 u'_1$. Then

$$ab = h_1 u_1 h_2 u_2 = h_1 (u_1 h_2) u_2 = h_1 (h_2 u'_1) u_2 = (h_1 h_2) (u'_1 u_2).$$

Here $h_1 h_2 \in H$ and $u'_1 u_2 \in U$, since H and U are subgroups. □

Lemma 3.2 (`hu_inv`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$ and $a \in G_c$ with $a = hu$ for some $h \in H$ and $u \in U$. Then there exist $h' \in H$ and $u' \in U$ with $a^{-1} = h' u'$.*

Lean: `hu_inv` — proved by AIProver, job 2, session 1

```

theorem hu_inv (c : ℕ → ℕ) (a : GM.Grp c)
  (ha : ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a = h * u) :
  ∃ h' ∈ GM.torusSub c, ∃ u' ∈ GM.unipSub c, a⁻¹ = h' * u' := by
  rcases ha with ⟨h, hh, u, hu, rfl⟩
  have h_inv : h⁻¹ ∈ GM.torusSub c := Subgroup.inv_mem _ hh
  have u_inv : u⁻¹ ∈ GM.unipSub c := Subgroup.inv_mem _ hu
  have h_comm := gsimple_torus_unipotent_commute c (h⁻¹) h_inv (u⁻¹) u_inv
  rcases h_comm with ⟨u', hu', h_eq⟩
  refine ⟨h⁻¹, h_inv, u', hu', ?_⟩
  calc
    (h * u)⁻¹ = u⁻¹ * h⁻¹ := by group
    _ = h⁻¹ * u' := h_eq

```

Proof. Since H and U are subgroups, $h^{-1} \in H$ and $u^{-1} \in U$. Apply `gsimple_torus_unipotent_commute` to h^{-1} and u^{-1} . This gives $u' \in U$ with $u^{-1}h^{-1} = h^{-1}u'$. Hence

$$a^{-1} = (hu)^{-1} = u^{-1}h^{-1} = h^{-1}u',$$

and we take $h' = h^{-1}$. □

Lemma 3.3 (`hu_one`). *For every $c: \mathbb{N} \rightarrow \mathbb{N}$, there exist $h \in H$ and $u \in U$ with $1 = hu$ in G_c .*

Lean: `hu_one` — proved by AIProver, job 2, session 1

```

theorem hu_one (c : ℕ → ℕ) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, (1 : GM.Grp c) = h * u := by
  have h1 : (1 : GM.Grp c) ∈ GM.torusSub c := Subgroup.one_mem _
  have h2 : (1 : GM.Grp c) ∈ GM.unipSub c := Subgroup.one_mem _
  refine ⟨1, h1, 1, h2, ?_⟩
  simp

```

Proof. Take $h = 1 \in H$ and $u = 1 \in U$. □

Lemma 3.4 (`hu_of_torus`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$ and $x \in H$. Then there exist $h \in H$ and $u \in U$ with $x = hu$.*

Lean: `hu_of_torus` — proved by AIProver, job 4, session 1

```

theorem hu_of_torus (c : ℕ → ℕ) (x : GM.Grp c) (hx : x ∈ GM.torusSub c) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, x = h * u := by
  refine ⟨x, hx, 1, Subgroup.one_mem _, ?_⟩
  simp

```

Proof. Take $h = x$ and $u = 1$. □

Lemma 3.5 (`hu_of_unip`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$ and $x \in U$. Then there exist $h \in H$ and $u \in U$ with $x = hu$.*

Lean: `hu_of_unip` — proved by AIProver, job 2, session 0

```

theorem hu_of_unip (c : ℕ → ℕ) (x : GM.Grp c) (hx : x ∈ GM.unipSub c) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, x = h * u := by
  refine ⟨1, Subgroup.one_mem _, x, hx, ?_⟩
  simp

```

Proof. Take $h = 1$ and $u = x$. □

Lemma 3.6 (`hu_gen`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$ and let $g \in \text{Gen}_c$ be any symbol. Then there exist $h \in H$ and $u \in U$ such that the image of g in G_c equals hu .*

Lean: hu_gen — proved by AIProver, job 2, session 0

```

theorem hu_gen (c : ℕ → ℕ) (g : GM.Gen c) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c,
    (PresentedGroup.of g : GM.Grp c) = h * u := by
  induction g with
  | H1 s =>
    refine ⟨PresentedGroup.of (GM.Gen.H1 s), Subgroup.subset_closure (by
      refine ⟨s, Or.inl rfl⟩), 1, Subgroup.one_mem _, ?_⟩
    rfl
  | H2 s =>
    refine ⟨PresentedGroup.of (GM.Gen.H2 s), Subgroup.subset_closure (by
      refine ⟨s, Or.inr rfl⟩), 1, Subgroup.one_mem _, ?_⟩
    rfl
  | Xm u =>
    refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.Xm u),
      Subgroup.subset_closure (Or.inl ⟨u, rfl⟩), ?_⟩
    rfl
  | Ym u =>
    refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.Ym u),
      Subgroup.subset_closure (Or.inr (Or.inl ⟨u, rfl⟩)), ?_⟩
    rfl
  | X l j k h u =>
    refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.X l j k h u),
      Subgroup.subset_closure (Or.inr (Or.inr (Or.inl ⟨1, j, k, h, u, rfl⟩))), ?_⟩
    rfl
  | Y l j k h u =>
    refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.Y l j k h u),
      Subgroup.subset_closure (Or.inr (Or.inr (Or.inr ⟨1, j, k, h, u, rfl⟩))), ?_⟩
    rfl

```

Proof. We argue by cases on the kind of symbol.

- If $g = H_1(s)$ or $g = H_2(s)$ with $s \in \mathbb{C}^\times$, its image lies in $\text{torusGens}_c \subseteq H$. Take h to be this image and $u = 1$.
- If g is one of $X_{-1}(u_0)$, $Y_{-1}(u_0)$, $X_{\ell,jk}(u_0)$, $Y_{\ell,jk}(u_0)$, its image lies in $\text{unipGens}_c \subseteq U$. Take $h = 1$ and u to be this image.

In each case the required equation holds by definition. □

3.2 Proof of the theorem

Lean: solution — main proof from the captain's sketch

```

theorem solution (c : ℕ → ℕ) :
  ∀ g : GM.Grp c, ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, g = h * u := by
  intro g
  let S : Set (GM.Grp c) := {x | ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, x = h * u}
  let N : Subgroup (GM.Grp c) :=
    { carrier := S
      mul_mem' := fun {a b} ha hb => hu_mul c a b ha hb
      one_mem' := hu_one c
      inv_mem' := fun {a} ha => hu_inv c a ha }
  have hN : N = ⊤ := subgroup_top_of_gens c N (fun x => hu_gen c x)

```

```

have hg : g ∈ N := by
  rw [hN]
  exact Subgroup.mem_top g
exact hg

```

Proof. Fix c and let $S = HU = \{x \in G_c \mid \exists h \in H, u \in U : x = hu\}$. By [Lemma 3.1](#), [Lemma 3.3](#) and [Lemma 3.2](#), S is closed under multiplication, contains 1, and is closed under inversion. So S is the carrier of a subgroup $N \leq G_c$.

By [Lemma 3.6](#), N contains the image of every generating symbol. The library result `subgroup_top_of_gens` then gives $N = G_c$.

Hence every $g \in G_c$ lies in S , that is, $g = hu$ for some $h \in H$ and $u \in U$. This proves $G_c = HU$. $\square$

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions.

- `GM.Valid` encodes the index set E : the condition $\ell < j$, $1 \leq k \leq c(j)$. Here $c : \mathbb{N} \rightarrow \mathbb{N}$ is a parameter, which the formalization note permits. Since $\ell < j$ is required, $j \geq 1$ automatically.
- `GM.Gen` encodes the symbol set $\mathcal{X}$: $H_1(s)$, $H_2(s)$ for $s \in \mathbb{C}^\times$; $X_{-1}(u)$, $Y_{-1}(u)$ for $u \in \mathbb{C}$; and $X_{\ell,jk}(u)$, $Y_{\ell,jk}(u)$ for valid triples. `GM.F` is the free group on it.
- `GM.cc` matches $c_{\ell j} = (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell)$. The natural-number subtraction $j - \ell$ is harmless because $\ell < j$.
- `GM.wm` and `GM.ww` match `(Re:0)` and `(Im:0)`.
- `GM.cm` is $aba^{-1}b^{-1}$. `GM.eqr` $a b^{-1}$ is the relator for $a = b$.
- `GM.Rel`: I compared every constructor with the source.
 - `(H:1a)`, `(H:1b)`, `(H:2)`, `(Re:1a)`–`(Re:6d)` and `(Im:1a)`–`(Im:6d)` agree with the source. This includes the inverted exponents in `(Im:5a)`, `(Im:5b)`, `(Im:6c)`, `(Im:6d)` and the $c_{\ell j}$ factors in `(Im:2)` and `(Im:4)`.
 - `(U:1a)`–`(U:1d)` are stated with validity witnesses. For $j = 0$, the side condition `Valid(j - 1, j, k)` is false, which matches the source because there is no such element of E .
 - `(U:2)` uses the disjunction $j \neq p \vee k \neq q \vee |\ell - m| > 1$, as in the source.
 - `(U:3a)`, `(U:3b)`, `(U:4a)`, `(U:4b)` agree with the source. In `(U:4)` the extra witness for $(j - 1 - \ell, j, k)$ is always available when $(\ell, j, k) \in E$, so no instance is lost.
- `GM.Grp` is the presented group $F(\mathcal{X})/N_{\mathcal{R}}$.
- `GM.torusSub` is the closure of the images of $H_1(s)$ and $H_2(s)$, i.e. H .
- `GM.unipSub` is the closure of the images of all unipotent generators, i.e. U .

No discrepancies found. The relation set is not degenerate.

Statement. The target is: for every c and every $g \in G(\mathfrak{m})$ there exist $h \in H$ and $u \in U$ with $g = hu$. This says exactly $G(\mathfrak{m}) \subseteq HU$; the reverse inclusion is trivial. The source's equality $G(\mathfrak{m}) = HU$ therefore follows.

No hypothesis on c is assumed, so the statement is more general than the source. The source's statement is recovered by taking $c(j)$ to be the coefficients of J for $j \geq 1$; the value $c(0)$ is irrelevant because $j \geq 1$ is forced.

Proof. The proof follows the source proof directly.

- `hu_mul` gives closure of HU under multiplication via $h_1u_1h_2u_2 = (h_1h_2)(u'_1u_2)$.
- `hu_inv` gives closure under inversion via $(hu)^{-1} = u^{-1}h^{-1} = h^{-1}u'$.
- `hu_one` shows $1 \in HU$.
- These three lemmas package HU as a subgroup N .
- `hu_gen` shows that every generator lies in N .
- `subgroup_top_of_gens`, which uses `PresentedGroup.closure_range_of`, gives $N = \mathbb{T}$.

Lemma 3.5 (`gsimple_torus_unipotent_commute`) is proved in the file, not assumed. It rests on Theorem 3.4 (`gsimple_torus_normalizes_unipotent`), which is also proved in the file:

- relations (Re:6) and (Im:6) show that conjugating a unipotent generator by $H_1(s)^{\pm 1}$ or $H_2(s)^{\pm 1}$ gives a unipotent generator;
- closure induction then extends this to all of H and U ;
- inverses of torus generators are handled by `h1_inv_mk` and `h2_inv_mk`, derived from (H:1a) and (Re:5a).

The perfectness results from Theorem 3.3 in the file are not used by `solution`, apart from the auxiliary lemmas `h2_eq_conj` and `subgroup_top_of_gens`. Neither of these needs the hypothesis $c(2) \geq 1$.

Concerns. None

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

set_option autoImplicit false

-- From run srv_20261004_123235_GSimple_Theorem_3_3 (GSimple_Theorem_3_3), proved.

namespace GM

/-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
def Valid (c : ℕ → ℕ) (l j k : ℕ) : Prop := l < j ∧ 1 ≤ k ∧ k ≤ c j
```

```

/-- The generating symbols. -/
inductive Gen (c : ℕ → ℕ) : Type
| H1 : ℂ× → Gen c
| H2 : ℂ× → Gen c
| Xm : ℂ → Gen c
| Ym : ℂ → Gen c
| X (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : Gen c
| Y (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : Gen c

abbrev F (c : ℕ → ℕ) := FreeGroup (Gen c)

def h1 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H1 s)
def h2 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H2 s)
def xm {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Xm u)
def ym {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Ym u)
def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)

/-- c_{ℓ j} = (-1)^(ℓ+1) * binom(j-1,ℓ) * (ℓ+1) * (j-ℓ). -/
noncomputable def cc (l j : ℕ) : ℂ :=
  (-1 : ℂ) ^ (l + 1) * (((j - 1).choose l : ℕ) : ℂ) * ((l : ℂ) + 1) * ((j - 1 : ℕ) : ℂ)

/-- commutator (a,b) = a b a-1 b-1 -/
def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a-1 * b-1

/-- relator for the equation a = b -/
def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b-1

noncomputable def wm {c : ℕ → ℕ} (s : ℂ×) : F c :=
  xm (s : ℂ) * ym (-(s : ℂ)-1) * xm (s : ℂ)

noncomputable def ww {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
  xx l j k h (s : ℂ) * yy l j k h (-(s : ℂ)-1) / cc l j * xx l j k h (s : ℂ)

/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂ×) : Rel c (eqr (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂ×) : Rel c (eqr (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂ×) : Rel c (eqr (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqr (xm u * xm v) (xm (u + v)))
| Re1b (u v : ℂ) : Rel c (eqr (ym u * ym v) (ym (u + v)))
| Re2 (s t : ℂ×) : Rel c (eqr (ym (-(t : ℂ)) * xm (s : ℂ) * ym (t : ℂ))
  (xm (-(t : ℂ)-1) * ym (-(t : ℂ) ^ 2 * (s : ℂ) * xm ((t : ℂ)-1)))
| Re3 (s : ℂ×) : Rel c (eqr (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqr (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u : ℂ) : Rel c (eqr (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s : ℂ×) : Rel c (eqr (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s : ℂ×) : Rel c (eqr (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s : ℂ×) (u : ℂ) : Rel c (eqr (h1 s * xm u * (h1 s)-1) (xm ((s : ℂ) * u)))
| Re6b (s : ℂ×) (u : ℂ) : Rel c (eqr (h2 s * xm u * (h2 s)-1) (xm ((s : ℂ)-1 * u)))
| Re6c (s : ℂ×) (u : ℂ) : Rel c (eqr (h1 s * ym u * (h1 s)-1) (ym ((s : ℂ)-1 * u)))
| Re6d (s : ℂ×) (u : ℂ) : Rel c (eqr (h2 s * ym u * (h2 s)-1) (ym ((s : ℂ) * u)))
| Im1a (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqr (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqr (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))
| Im2 (l j k : ℕ) (h : Valid c l j k) (s t : ℂ×) :

```

```

Rel c (eqr (yy 1 j k h (-(t : ℂ)) * xx 1 j k h (s : ℂ) * yy 1 j k h (t : ℂ))
  (xx 1 j k h (-(t : ℂ)-1 / cc 1 j) *
    yy 1 j k h (-(cc 1 j) * (t : ℂ) ^ 2 * (s : ℂ)) *
    xx 1 j k h ((t : ℂ)-1 / cc 1 j)))
| Im3 (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h (s ^ ((1 + 1) * (j - 1))) * ww 1 j k h 1)
    (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (1 + 1))))
| Im4a (1 j k : ℕ) (h : Valid c 1 j k) (u : ℂ) :
  Rel c (eqr (ww 1 j k h 1 * xx 1 j k h u * (ww 1 j k h 1)-1) (yy 1 j k h (-u / cc 1 j)))
| Im4b (1 j k : ℕ) (h : Valid c 1 j k) (u : ℂ) :
  Rel c (eqr (ww 1 j k h 1 * yy 1 j k h u * (ww 1 j k h 1)-1) (xx 1 j k h (-(cc 1 j) * u)))
| Im5a (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h 1 * h1 (s ^ (j - 1)) * (ww 1 j k h 1)-1) (h2 ((s ^ (1 + 1))-1)))
| Im5b (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h 1 * h2 (s ^ (1 + 1)) * (ww 1 j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * xx 1 j k h u * (h1 s)-1) (xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)))
| Im6b (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * xx 1 j k h u * (h2 s)-1) (xx 1 j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * yy 1 j k h u * (h1 s)-1) (yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)))
| Im6d (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * yy 1 j k h u * (h2 s)-1) (yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (1 j k : ℕ) (h : Valid c 1 j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ)
  (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((1 : ℤ) - (m : ℤ))) :
  Rel c (cm (xx 1 j k h u) (yy m p q h' v))
| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (1 j k : ℕ) (h : Valid c 1 j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * xx 1 j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (1 j k : ℕ) (h : Valid c 1 j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * yy 1 j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}

/-- The group G(m) = ⟨X | R⟩. -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

end GM

theorem mk_rel_one (c : ℕ → ℕ) (r : GM.F c) (h : GM.Rel c r) :
  PresentedGroup.mk (GM.Rels c) r = 1 := by
  apply PresentedGroup.one_of_mem
  exact h

```



```

theorem mk_eqr (c : ℕ → ℕ) (a b : GM.F c) (h : GM.Rel c (GM.eqr a b)) :
  PresentedGroup.mk (GM.Rels c) a = PresentedGroup.mk (GM.Rels c) b := by
  exact (PresentedGroup.mk_eq_mk_of_mul_inv_mem h)

theorem subgroup_top_of_gens (c : ℕ → ℕ) (N : Subgroup (GM.Grp c))
  (h : ∀ g : GM.Gen c, (PresentedGroup.of g : GM.Grp c) ∈ N) : N = ⊤ := by
  have hgen : Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c → GM.Grp c) ⊆
  ↪ N.carrier := by
    rintro _ ⟨g, rfl⟩
    exact h g
  have hclosure : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) :
  ↪ GM.Gen c → GM.Grp c)) ≤ N :=
    ((Subgroup.closure_le (K := N)).mpr hgen)
  have htop : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c
  ↪ → GM.Grp c)) = ⊤ :=
    PresentedGroup.closure_range_of (GM.Rels c)
  rw [htop] at hclosure
  exact top_le_iff.mp hclosure

theorem cm_mem_commutator (c : ℕ → ℕ) (a b : GM.F c) :
  PresentedGroup.mk (GM.Rels c) (GM.cm a b) ∈ commutator (GM.Grp c) := by
  have h : PresentedGroup.mk (GM.Rels c) (GM.cm a b) = [[PresentedGroup.mk (GM.Rels c) a,
  PresentedGroup.mk (GM.Rels c) b]] := by
    simp [GM.cm, commutatorElement_def]
  rw [h]
  exact Subgroup.commutator_mem_commutator (Subgroup.mem_top _) (Subgroup.mem_top _)

theorem xm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xm u) ∈ commutator (GM.Grp c) := by
  set G := GM.Grp c
  let m := PresentedGroup.mk (GM.Rels c)
  have hvalid1 : GM.Valid c 1 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hvalid0 : GM.Valid c 0 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hrel : GM.Rel c (GM.eqr (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) (GM.xm ((u :
  ↪ ℂ) * 1))) :=
    GM.Rel.U3a 1 hvalid1 hvalid0 (u : ℂ) 1
  have h_eq : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) = m (GM.xm ((u : ℂ) * 1)) := by
    apply mk_eqr c _ _ hrel
  have h_cm_mem : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) ∈ commutator G := by
    rw [commutator_eq_closure G]
    apply Subgroup.subset_closure
    exact commutator_mem_commutatorSet (m (GM.xx 1 2 1 hvalid1 (u : ℂ))) (m (GM.yy 0 2 1 hvalid0 1))
  have h_target : m (GM.xm u) = m (GM.xm ((u : ℂ) * 1)) := by simp
  rw [h_target]
  rw [← h_eq]
  exact h_cm_mem

theorem ym_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.ym u) ∈ commutator (GM.Grp c) := by
  have hvalid0 : GM.Valid c 0 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have hvalid1 : GM.Valid c 1 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc

```

```

have h_rel : GM.Rel c (GM.eqr (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1))
  (GM.ym (u * 1))) :=
  GM.Rel.U3b 1 hvalid0 hvalid1 u 1
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) =
  PresentedGroup.mk (GM.Rels c) (GM.ym (u * 1)) :=
  mk_eqr c _ _ h_rel
have h_cm_mem : PresentedGroup.mk (GM.Rels c)
  (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) ∈ commutator (GM.Grp c) :=
  cm_mem_commutator c _ _
rw [h_eq] at h_cm_mem
simp [mul_one] using h_cm_mem

theorem wm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) := by
have hx : PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) ∈ commutator (GM.Grp c) :=
  xm_mem c hc (s : ℂ)
have hy : PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) ∈ commutator (GM.Grp c) :=
  ym_mem c hc (-(s : ℂ)-1)
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s) =
  PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) *
  PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) *
  PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) := by
  simp [GM.wm, GM.xm, GM.ym, map_mul]
rw [h_eq]
exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hx hy) hx

theorem exists_unit_pow_ne_one (d : ℕ) (hd : 0 < d) :
  ∃ s : ℂ×, (s : ℂ) ^ d ≠ 1 := by
have h2val : (2 : ℂ) ^ d ≠ 1 := by
  intro h
  have hnormSq : Complex.normSq ((2 : ℂ) ^ d) = Complex.normSq (1 : ℂ) := by rw [h]
  have h_one : Complex.normSq (1 : ℂ) = 1 := by simp
  rw [h_one] at hnormSq
  have hcalc : Complex.normSq ((2 : ℂ) ^ d) = (Complex.normSq (2 : ℂ)) ^ d := by
    simp using map_pow Complex.normSq (2 : ℂ) d
  have h4 : Complex.normSq (2 : ℂ) = (4 : ℝ) := by norm_num
  rw [hcalc, h4] at hnormSq

  have h4pos : (0 : ℝ) < 4 := by norm_num
  have h4dpos : 0 < (4 : ℝ) ^ d := pow_pos h4pos d
  have h4dgt1 : (1 : ℝ) < (4 : ℝ) ^ d := by
    have h_nat : 1 < (4 : ℕ) ^ d := Nat.one_lt_pow (by omega : d ≠ 0) (by norm_num : 1 < 4)
    exact_mod_cast h_nat
  linarith
  refine ⟨Units.mk0 (2 : ℂ) (by norm_num), h2val⟩

theorem xx_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v))) :=
  GM.Rel.Im1a l j k h u v
have h_eq := mk_eqr c (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v)) hrel
calc
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v))
    = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u * GM.xx l j k h v) := by
    rw [← h_eq]
  _ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by

```

```

rw [map_mul]

theorem yy_add_mk (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) (h : GM.Valid c l j k) (u v :  $\mathbb{C}$ ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) *
      PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.yy l j k h u * GM.yy l j k h v) (GM.yy l j k h (u + v))) :=
  GM.Rel.Im1b l j k h u v
have := mk_eqr c (GM.yy l j k h u * GM.yy l j k h v) (GM.yy l j k h (u + v)) hrel
simpa [GM.eqr, mul_inv_rev] using this.symm

theorem xx_sub_mk (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) (h : GM.Valid c l j k) (a b :  $\mathbb{C}$ ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.xx l j k h b))-1 := by
have hzero : PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (0 :  $\mathbb{C}$ )) = 1 := by
have hrel : GM.Rel c (GM.eqr (GM.xx l j k h (0 :  $\mathbb{C}$ )) 1) := by
have := GM.Rel.Im1a l j k h (0 :  $\mathbb{C}$ ) (0 :  $\mathbb{C}$ )
simpa [GM.eqr] using this
have hmem : GM.eqr (GM.xx l j k h (0 :  $\mathbb{C}$ )) 1  $\in$  GM.Rels c := hrel
exact PresentedGroup.one_of_mem hmem
have hneg (u :  $\mathbb{C}$ ) : PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (-u)) =
  (PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u))-1 := by
have h := xx_add_mk c l j k h u (-u)

have hzero' : (u + (-u :  $\mathbb{C}$ )) = (0 :  $\mathbb{C}$ ) := by ring
rw [hzero'] at h
rw [hzero] at h

exact eq_inv_of_mul_eq_one_right h.symm
calc
PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (a - b))
= PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (a + (-b))) := by ring
_ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h a) *
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (-b)) := xx_add_mk c l j k h a (-b)
_ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h a) *
  (PresentedGroup.mk (GM.Rels c) (GM.xx l j k h b))-1 := by rw [hneg b]

theorem yy_sub_mk (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) (h : GM.Valid c l j k) (a b :  $\mathbb{C}$ ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy l j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h b))-1 := by
have hRel : GM.Rel c (GM.eqr (GM.yy l j k h (a - b) * GM.yy l j k h b) (GM.yy l j k h a)) := by
have := GM.Rel.Im1b l j k h (a - b) b
simpa [sub_add_cancel] using this
have hEq := mk_eqr c (GM.yy l j k h (a - b) * GM.yy l j k h b) (GM.yy l j k h a) hRel

have hMul : PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (a - b) * GM.yy l j k h b) =
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (a - b)) *
    PresentedGroup.mk (GM.Rels c) (GM.yy l j k h b) := by
simp
rw [hMul] at hEq

calc
PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (a - b))

```

```

    = (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
      PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b)) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
simp
_ = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
rw [hEq]

theorem xx_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) *
  (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
have him6b := GM.Rel.Im6b 1 j k h s v
have hmem : GM.eqr (GM.h2 s * GM.xx 1 j k h v * (GM.h2 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) ∈
↪ GM.Rels c := him6b
have heq := PresentedGroup.mk_eq_mk_of_mul_inv_mem hmem
rw [← heq]
simp [map_mul, map_inv, commutatorElement_def] using Subgroup.commutator_mem_commutator
(g1 := PresentedGroup.mk (GM.Rels c) (GM.h2 s))
(g2 := PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))
(h1 := show PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ (⊤ : Subgroup (GM.Grp c)) from Subgroup.mem_top _)
(h2 := show PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) ∈ (⊤ : Subgroup (GM.Grp c)) from
↪ Subgroup.mem_top _)

open GM in
theorem yy_conj_mem_aiprover_s0 (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
have h_rel := Rel.Im6d 1 j k h s v
have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1)
  (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) h_rel
have h_mk : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
  (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
  (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by
calc
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
    PresentedGroup.mk (GM.Rels c) (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1) := h_eq.symm
  _ = (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by simp [map_mul]
rw [h_mk]
have h_comm : (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
  (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)-1 =
  [[(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c),
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)]] := rfl
rw [h_comm]
dsimp [commutator]
apply Subgroup.subset_closure
exact ⟨(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c), Subgroup.mem_top _,
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c), Subgroup.mem_top _, rfl⟩

theorem yy_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
first
| exact yy_conj_mem_aiprover_s0
| (intros; apply yy_conj_mem_aiprover_s0 <;> assumption)

```

```

theorem xx_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) ∈ commutator (GM.Grp c) := by
  have hpos_or : j - 1 = 0 ∨ 0 < j - 1 := Nat.eq_zero_or_pos (j - 1)
  rcases hpos_or with (hjl | hdpos)
  · rcases h with ⟨hlt, _, _⟩
    have : 1 < 1 :=
      Nat.lt_of_lt_of_le hlt (Nat.le_of_sub_eq_zero hjl)
    exact absurd this (Nat.lt_irrefl _)
  · rcases exists_unit_pow_ne_one (j - 1) hdpos with ⟨s, hs⟩
    have hden_ne_zero : (s : ℂ) ^ (j - 1) - 1 ≠ 0 := by
      intro hzero
      have h_one : (s : ℂ) ^ (j - 1) = 1 := sub_eq_zero.mp hzero
      exact hs h_one
    set v := u / ((s : ℂ) ^ (j - 1) - 1) with hv_def
    have h_eq : u = ((s : ℂ) ^ (j - 1) - 1) * v := by
      dsimp [v]
      field_simp [hden_ne_zero]
    rw [h_eq]
    have : ((s : ℂ) ^ (j - 1) - 1) * v = ((s : ℂ) ^ (j - 1)) * v - v := by ring
    rw [this]
    rw [xx_sub_mk c l j k h (((s : ℂ) ^ (j - 1)) * v) v]
    exact xx_conj_mem c l j k h s v

theorem yy_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) ∈ commutator (GM.Grp c) := by
  have hjl : 0 < j - 1 := by
    have := h.1
    omega
  obtain ⟨s, hs⟩ := exists_unit_pow_ne_one (j - 1) hjl
  have ht : ((s : ℂ) ^ (j - 1))-1 - 1 ≠ 0 := by
    intro h0
    apply hs
  have h1 : ((s : ℂ) ^ (j - 1))-1 = 1 := sub_eq_zero.mp h0
  first
    | exact inv_eq_one.mp h1
    | simpa using h1
  have hu : ((s : ℂ) ^ (j - 1))-1 * (u / (((s : ℂ) ^ (j - 1))-1 - 1)) -
    u / (((s : ℂ) ^ (j - 1))-1 - 1) = u := by
    rw [← sub_one_mul]
    first
      | exact mul_div_cancel0 u ht
      | (rw [mul_comm]; exact div_mul_cancel0 u ht)
      | field_simp
  have key := yy_conj_mem c l j k h s (u / (((s : ℂ) ^ (j - 1))-1 - 1))
  first
    | (rw [← yy_sub_mk c l j k h, hu] at key; exact key)
    | (rw [← hu, yy_sub_mk c l j k h]; exact key)

theorem ww_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.ww l j k h s) ∈ commutator (GM.Grp c) := by
  let f : GM.F c →* GM.Grp c := PresentedGroup.mk (GM.Rels c)
  have hxx : f (GM.xx l j k h (s : ℂ)) ∈ commutator (GM.Grp c) := by
    dsimp [f]
    simpa using xx_mem c l j k h (s : ℂ)
  have hyy : f (GM.yy l j k h (-(s : ℂ)-1) / GM.cc l j) ∈ commutator (GM.Grp c) := by
    simpa using yy_mem c l j k h (-(s : ℂ)-1) / GM.cc l j)

```

```

have hprod : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j k h (s : ℂ)
↪ ∈ commutator (GM.Grp c) := by
  have hcalc : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j k h (s :
↪ ℂ)) =
    f (GM.xx 1 j k h (s : ℂ)) * f (GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * f (GM.xx 1 j k h (s :
↪ ℂ)) := by
    simp [map_mul]
    rw [hcalc]
    exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hxx hyy) hxx

have hww : GM.ww 1 j k h s = GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j
↪ k h (s : ℂ) := rfl
rw [hww]
exact hprod

theorem h1h2_neg_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) ∈ commutator (GM.Grp c) := by
  have hrel : GM.Rel c (GM.eqr (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1)))) :=
    GM.Rel.Re3 s
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) :=
    mk_eqr c (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1))) hrel
  have h_wm_s : PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) :=
    wm_mem c hc s
  have h_wm_1 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) :=
    wm_mem c hc 1
  have h_prod : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) ∈ commutator (GM.Grp c) :=
    Subgroup.mul_mem _ h_wm_s h_wm_1
  rw [← h_eq]
  exact h_prod

theorem h1h2_inv_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (x : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 x * GM.h2 x-1) ∈ commutator (GM.Grp c) := by
  have hrel := GM.Rel.Re3 (c := c) (-x)
  have h_simplified : (GM.h1 (c := c) (-(-x)) * GM.h2 (c := c) (-(x-1))) = (GM.h1 (c := c) x * GM.h2 (c
↪ := c) x-1) := by
    simp
  have hmem : PresentedGroup.mk (GM.Rels c) (GM.wm (-x) * GM.wm 1) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) (GM.wm (-x)) ∈ commutator (GM.Grp c) := wm_mem c hc (-x)
    have h2 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
    exact Subgroup.mul_mem _ h1 h2
  have heq := mk_eqr c (GM.wm (-x) * GM.wm 1) (GM.h1 x * GM.h2 x-1) (by
    simpa [h_simplified] using hrel)
  rw [← heq]
  exact hmem

theorem im3_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c 1 j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))
  ∈ commutator (GM.Grp c) := by
  set w1 := GM.ww 1 j k h (s ^ ((1 + 1) * (j - 1))) with hw1
  set w0 := GM.ww 1 j k h 1 with hw0
  have hIm3_rel : GM.Rel c (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))) :=
    GM.Rel.Im3 1 j k h s
  have h_mul_inv : w1 * w0 * (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))-1 ∈ GM.Rels c := by
    simpa [GM.Rels] using hIm3_rel
  have h_mem' : PresentedGroup.mk (GM.Rels c) (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 +
↪ 1)))) = 1 :=
    PresentedGroup.one_of_mem hIm3_rel
  have h_eq : PresentedGroup.mk (GM.Rels c) (w1 * w0) =

```

```

    PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1))) := by
    apply PresentedGroup.mk_eq_mk_of_mul_inv_mem h_mul_inv
  have h_prod_mem : PresentedGroup.mk (GM.Rels c) (w1 * w0) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) w1 ∈ commutator (GM.Grp c) :=
      ww_mem c 1 j k h (s ^ ((1 + 1) * (j - 1)))
    have h2 : PresentedGroup.mk (GM.Rels c) w0 ∈ commutator (GM.Grp c) :=
      ww_mem c 1 j k h 1
    exact Subgroup.mul_mem (commutator (GM.Grp c)) h1 h2
  rw [← h_eq]
  exact h_prod_mem

theorem h1sq_h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (y : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) ∈ commutator (GM.Grp c) := by
  have hvalid : GM.Valid c 0 2 1 := ⟨by decide, by decide, hc⟩
  have hw1 := ww_mem c 0 2 1 hvalid (y ^ 2)
  have hw2 := ww_mem c 0 2 1 hvalid 1
  have hprod : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) ∈ commutator (GM.Grp c) :=
    (commutator (GM.Grp c)).mul_mem hw1 hw2
  have him3 : GM.Rel c (GM.eqr (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1)
    (GM.h1 (y ^ 2) * GM.h2 y)) := by
    have := GM.Rel.Im3 0 2 1 hvalid (-y)
    simpa [show ((-y : ℂ×) : ℂ) ^ ((0+1)*(2-0)) = (y : ℂ)^2 by norm_num] using this
  have heq : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) := by
    simpa [GM.Rels] using PresentedGroup.mk_eq_mk_of_mul_inv_mem him3
  rw [← heq]
  exact hprod

theorem h1_cube_eq (c : ℕ → ℕ) (z : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) =
  PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 z-1))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x

  have h1_mul : ∀ s t : ℂ×, φ (GM.h1 (c := c) (s * t)) = φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) t) := by
    intro s t
    have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) :=
      GM.Rel.H1a (c := c) s t
    have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

  have hcalc : φ (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) =
    φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1 := by
    simp [GM.eqr, hφ_mul, hφ_inv]
  rw [hcalc] at h_one

  have h_eq : φ (GM.h1 (c := c) s * GM.h1 (c := c) t) = φ (GM.h1 (c := c) (s * t)) := by

```

```

    calc
       $\varphi (GM.h1 (c := c) s * GM.h1 (c := c) t) =$ 
       $(\varphi (GM.h1 (c := c) s * GM.h1 (c := c) t) * (\varphi (GM.h1 (c := c) (s * t)))^{-1}) *$ 
       $\varphi (GM.h1 (c := c) (s * t)) := \text{by group}$ 
       $_ = 1 * \varphi (GM.h1 (c := c) (s * t)) := \text{by rw [h\_one]}$ 
       $_ = \varphi (GM.h1 (c := c) (s * t)) := \text{by simp}$ 
    rw [h $\varphi$ _mul] at h_eq
    exact h_eq.symm

have h1_one :  $\varphi (GM.h1 (c := c) 1) = 1 := \text{by}$ 
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) :=
    GM.Rel.H1a (c := c) 1 1
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

have hcalc :  $\varphi (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) =$ 
   $\varphi (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) * (\varphi (GM.h1 (c := c) (1 * 1)))^{-1} := \text{by}$ 
  simp [GM.eqr, h $\varphi$ _mul]
  rw [hcalc] at h_one

rw [mul_one] at h_one

rw [h $\varphi$ _mul] at h_one

calc
   $\varphi (GM.h1 (c := c) 1) = (\varphi (GM.h1 (c := c) 1) * \varphi (GM.h1 (c := c) 1)) * (\varphi (GM.h1 (c := c) 1))^{-1} := \text{by}$ 
   $\hookrightarrow \text{group}$ 
   $_ = 1 := h\_one$ 

have h1_inv :  $\forall s : \mathbb{C}^\times, \varphi (GM.h1 (c := c) s^{-1}) = (\varphi (GM.h1 (c := c) s))^{-1} := \text{by}$ 
  intro s
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) (GM.h1 (c := c) (s * s^{-1}))) :=
    GM.Rel.H1a (c := c) s s^{-1}

  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
  have hcalc :  $\varphi (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) (GM.h1 (c := c) (s * s^{-1}))) =$ 
     $\varphi (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) * (\varphi (GM.h1 (c := c) (s * s^{-1})))^{-1} := \text{by}$ 
    simp [GM.eqr, h $\varphi$ _mul, h $\varphi$ _inv]
  rw [hcalc] at h_one

have h_s_one :  $\varphi (GM.h1 (c := c) (s * s^{-1})) = 1 := \text{by}$ 
  have :  $s * s^{-1} = (1 : \mathbb{C}^\times) := \text{by simp}$ 
  rw [this]
  exact h1_one
  rw [h_s_one] at h_one

have :  $(1 : \text{PresentedGroup } (GM.Rels c))^{-1} = 1 := \text{by simp}$ 
  rw [this, mul_one] at h_one

calc
   $\varphi (GM.h1 (c := c) s^{-1}) = 1 * \varphi (GM.h1 (c := c) s^{-1}) := \text{by simp}$ 

```



```

_ = ((φ (GM.h1 (c := c) s))-1 * φ (GM.h1 (c := c) s)) * φ (GM.h1 (c := c) s-1) := by simp
_ = (φ (GM.h1 (c := c) s))-1 * (φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) s-1)) := by group
_ = (φ (GM.h1 (c := c) s))-1 * 1 := by
  rw [show φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) s-1) = 1 from h_one]
_ = (φ (GM.h1 (c := c) s))-1 := by simp

have h_pow_h1 : ∀ (x : ℂ×) (n : ℕ), φ (GM.h1 (c := c) (x ^ n)) = (φ (GM.h1 (c := c) x)) ^ n := by
  intro x n
  induction' n with k ih
  .
  simp [h1_one]
  .
  rw [pow_succ, pow_succ]

  rw [h1_mul (x ^ k) x]
  rw [ih]

have h_inv_h1 : ∀ (x : ℂ×), φ (GM.h1 (c := c) x-1) = (φ (GM.h1 (c := c) x))-1 := h1_inv

calc
  φ (GM.h1 (c := c) (z ^ 3)) = (φ (GM.h1 (c := c) z)) ^ 3 := by rw [h_pow_h1]
  _ = (φ (GM.h1 (c := c) z)) * ((φ (GM.h1 (c := c) z))-1 ^ 2)-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * (((φ (GM.h1 (c := c) z))-1 ^ 2) * φ (GM.h2 (c :=
→ c) z-1))-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * ((φ (GM.h1 (c := c) (z-1 ^ 2)) * φ (GM.h2 (c :=
→ c) z-1))-1) := by
  simp [h_pow_h1, h_inv_h1]
  _ = PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1 ^ 2) * GM.h2 z-1))-1 := by
  simp [φ, hφ_mul]

theorem exists_unit_cube (s : ℂ×) : ∃ z : ℂ×, z ^ 3 = s := by
  obtain ⟨w, hw⟩ := IsAlgClosed.exists_pow_nat_eq (s : ℂ) (by decide : 0 < 3)
  have hw_ne_zero : w ≠ 0 := by
    intro hzero
    rw [hzero, zero_pow (by decide : 3 ≠ 0)] at hw
  have hs_ne_zero : (s : ℂ) ≠ 0 := Units.ne_zero s
  exact hs_ne_zero (Eq.symm hw)
  let z : ℂ× := Units.mk0 w hw_ne_zero
  refine ⟨z, ?_⟩
  apply Units.ext_iff.mpr
  calc
    (z : ℂ) ^ 3 = w ^ 3 := rfl
    _ = (s : ℂ) := hw

theorem h1_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := by
  rcases exists_unit_cube s with ⟨z, hz⟩

have h_comm_z : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) ∈ commutator (GM.Grp c) :=
  h1h2_inv_mem c hc z

```

```

have h_comm_z_inv : PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)) ∈
  commutator (GM.Grp c) :=
  h1sq_h2_mem c hc (z-1)

have h_cube := h1_cube_eq c z

have h_prod : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)))-1 ∈
  commutator (GM.Grp c) := by
  apply Subgroup.mul_mem
  · exact h_comm_z
  · apply Subgroup.inv_mem
    exact h_comm_z_inv

have h_cube_mem : PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) ∈ commutator (GM.Grp c) := by
  rw [h_cube]
  exact h_prod

simp [hz] using h_cube_mem

theorem h2_eq_conj (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) =
  PresentedGroup.mk (GM.Rels c) (GM.wm 1) * PresentedGroup.mk (GM.Rels c) (GM.h1 s) *
  (PresentedGroup.mk (GM.Rels c) (GM.wm 1))-1 := by

have h_eq := mk_eqr c (GM.wm 1 * GM.h1 s * (GM.wm 1)-1) (GM.h2 s) (by
  apply GM.Rel.Re5a)

simp [PresentedGroup.mk, map_mul, map_inv] using h_eq.symm

theorem h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ commutator (GM.Grp c) := by
have hnormal : (commutator (GM.Grp c)).Normal := inferInstance
have hh1 : PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := h1_mem c hc s
have hww : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
rw [h2_eq_conj c s]
exact hnormal.conj_mem (PresentedGroup.mk (GM.Rels c) (GM.h1 s)) hh1
  (PresentedGroup.mk (GM.Rels c) (GM.wm 1))

theorem gsimple_g_m_perfect (c : ℕ → ℕ) (hc : 1 ≤ c 2) :
  commutator (GM.Grp c) = 1 := by
  apply subgroup_top_of_gens
  intro g
  cases g with
  | H1 s => exact h1_mem c hc s
  | H2 s => exact h2_mem c hc s
  | Xm u => exact xm_mem c hc u
  | Ym u => exact ym_mem c hc u
  | X l j k h u => exact xx_mem c l j k h u
  | Y l j k h u => exact yy_mem c l j k h u

-- From run srv_20261006_225233_GSimple_Theorem_3_4 (GSimple_Theorem_3_4), proved.

namespace GM

```

```

/-- Generators of the toral subgroup: the images of  $H_1(s)$  and  $H_2(s)$  in  $G(m)$ . -/
def torusGens (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Set (Grp c) :=
  {g |  $\exists s : \mathbb{C}^\times$ , g = PresentedGroup.mk (Rels c) (h1 s)  $\vee$ 
    g = PresentedGroup.mk (Rels c) (h2 s)}

/-- The toral subgroup H of  $G(m)$ . -/
def torusSub (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Subgroup (Grp c) := Subgroup.closure (torusGens c)

/-- The unipotent generators  $X_{\{-1\}}(u)$ ,  $Y_{\{-1\}}(u)$ ,  $X_{\{\ell,jk\}}(u)$ ,  $Y_{\{\ell,jk\}}(u)$ , viewed in  $G(m)$ . -/
def unipGens (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Set (Grp c) :=
  {g | ( $\exists u : \mathbb{C}$ , g = PresentedGroup.mk (Rels c) (xm u))  $\vee$ 
    ( $\exists u : \mathbb{C}$ , g = PresentedGroup.mk (Rels c) (ym u))  $\vee$ 
    ( $\exists (l j k : \mathbb{N})$  (h : Valid c l j k) (u :  $\mathbb{C}$ ),
      g = PresentedGroup.mk (Rels c) (xx l j k h u))  $\vee$ 
    ( $\exists (l j k : \mathbb{N})$  (h : Valid c l j k) (u :  $\mathbb{C}$ ),
      g = PresentedGroup.mk (Rels c) (yy l j k h u))}

/-- The unipotent subgroup U of  $G(m)$ . -/
def unipSub (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Subgroup (Grp c) := Subgroup.closure (unipGens c)

end GM

theorem h1_conj_unip (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (s :  $\mathbb{C}^\times$ ) (g : GM.Grp c) (hg : g  $\in$  GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1
   $\in$  GM.unipGens c := by

rcases hg with (<u, hg_u> | <u, hg_u> | <l, j, k, h, u, hg_u> | <l, j, k, h, u, hg_u>)
.
  subst hg_u

  have hrel := GM.Rel.Re6a (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xm u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u)) := by
    simp [GM.eqr] using mk_eqr c (GM.h1 s * GM.xm u * (GM.h1 s)-1) (GM.xm ((s :  $\mathbb{C}$ ) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u))  $\in$  GM.unipGens c := by
    refine Or.inl ((s :  $\mathbb{C}$ ) * u, ?_)
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u)) := by
    simp [GM.xm, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.

  subst hg_u

  have hrel := GM.Rel.Re6c (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.ym u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u)) := by
    simp [GM.eqr] using mk_eqr c (GM.h1 s * GM.ym u * (GM.h1 s)-1) (GM.ym (((s :  $\mathbb{C}$ )-1) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u))  $\in$  GM.unipGens c := by
    refine Or.inr (Or.inl (((s :  $\mathbb{C}$ )-1) * u, ?_))
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u))
   $\Rightarrow$  := by
    simp [GM.ym, map_mul, map_inv] using h_eq

```

```

rw [h_eq']
exact h_target
.

subst hg_u

have hrel := GM.Rel.Im6a (c := c) l j k h s u
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xx l j k h u * (GM.h1 s)-1) =
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h ((s : ℂ) ^ (1 + 1) * u)) := by
  simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.xx l j k h u * (GM.h1 s)-1) (GM.xx l j k h ((s : ℂ) ^ (1 +
→ 1) * u)) hrel
have h_target : PresentedGroup.mk (GM.Rels c) (GM.xx l j k h ((s : ℂ) ^ (1 + 1) * u)) ∈ GM.unipGens c
→ := by
  refine Or.inr (Or.inr (Or.inl ⟨l, j, k, h, ((s : ℂ) ^ (1 + 1) * u), ?_⟩))
  rfl
have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h ((s : ℂ) ^
→ (1 + 1) * u)) := by
  simpa [GM.xx, map_mul, map_inv] using h_eq
rw [h_eq']
exact h_target
.

subst hg_u

have hrel := GM.Rel.Im6c (c := c) l j k h s u
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.yy l j k h u * (GM.h1 s)-1) =
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (1 + 1))-1 * u)) := by
  simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.yy l j k h u * (GM.h1 s)-1) (GM.yy l j k h (((s : ℂ) ^ (1
→ + 1))-1 * u)) hrel
have h_target : PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (1 + 1))-1 * u)) ∈ GM.unipGens
→ c := by
  refine Or.inr (Or.inr (Or.inr ⟨l, j, k, h, (((s : ℂ) ^ (1 + 1))-1 * u), ?_⟩))
  rfl
have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ)
→ ^ (1 + 1))-1 * u)) := by
  simpa [GM.yy, map_mul, map_inv] using h_eq
rw [h_eq']
exact h_target

theorem h2_conj_unip (c : ℕ → ℕ) (s : ℂ×) (g : GM.Grp c) (hg : g ∈ GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1
  ∈ GM.unipGens c := by
  rcases hg with ⟨u, hg'⟩ | ⟨u, hg'⟩ | ⟨l, j, k, h, u, hg'⟩ | ⟨l, j, k, h, u, hg'⟩
  .
  subst hg'
  have h_rel := GM.Rel.Re6b (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xm u * (GM.h2 s)-1) (GM.xm ((s : ℂ)-1 * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ)-1 * u))
  → := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact Set.mem_setOf.mpr (Or.inl ⟨(s : ℂ)-1 * u, rfl⟩)
  .
  subst hg'
  have h_rel := GM.Rel.Re6d (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.ym u * (GM.h2 s)-1) (GM.ym ((s : ℂ) * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym ((s : ℂ) * u)) := by

```

```

    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inl ⟨(s : ℂ) * u, rfl⟩))

  subst hg'
  have h_rel := GM.Rel.Im6b (c := c) 1 j k h s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xx 1 j k h u * (GM.h2 s)-1) (GM.xx 1 j k h (((s : ℂ) ^ (j - 1)) * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (((s : ℂ)
  ^ (j - 1)) * u)) := by
    simp [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inl ⟨1, j, k, h, ((s : ℂ) ^ (j - 1)) * u, rfl⟩)))

  subst hg'
  have h_rel := GM.Rel.Im6d (c := c) 1 j k h s u
  have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h u * (GM.h2 s)-1) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1
  * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ)
  ^ (j - 1))-1 * u)) := by
    simp [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inr ⟨1, j, k, h, (((s : ℂ) ^ (j - 1))-1 * u, rfl⟩)))

theorem h1_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x
  have h1_one : φ (GM.h1 1) = 1 := by
    have hRel : GM.Rel c (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) :=
      GM.Rel.H1a (c := c) 1 1
    have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
    have hcalc : φ (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) =
      φ (GM.h1 1 * GM.h1 1) * (φ (GM.h1 (1 * 1)))-1 := by
      simp [GM.eqr, hφ_mul, hφ_inv]
    rw [hcalc] at h_one
    rw [mul_one] at h_one
    rw [hφ_mul] at h_one
    calc
      φ (GM.h1 1) = (φ (GM.h1 1) * φ (GM.h1 1)) * (φ (GM.h1 1))-1 := by group
      _ = 1 := h_one
  have hRel : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1
  have hRel' : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 1)) := by
    simp using hRel
  have h_eq := mk_eqr c (GM.h1 s * GM.h1 s-1) (GM.h1 1) hRel'
  calc
    φ (GM.h1 s-1) = 1 * φ (GM.h1 s-1) := by simp
    _ = ((φ (GM.h1 s))-1 * φ (GM.h1 s)) * φ (GM.h1 s-1) := by simp
    _ = (φ (GM.h1 s))-1 * (φ (GM.h1 s) * φ (GM.h1 s-1)) := by group
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 s * GM.h1 s-1) := by rw [hφ_mul]
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 1) := by rw [h_eq]
    _ = (φ (GM.h1 s))-1 * 1 := by rw [h1_one]
    _ = (φ (GM.h1 s))-1 := by simp

```

```

theorem h2_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 := by
  rw [h2_eq_conj c s-1, h2_eq_conj c s]
  simp
  rw [h1_inv_mk c s]
  group

theorem torusGens_conj (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, ∀ g ∈ GM.unipGens c, t * g * t-1 ∈ GM.unipGens c := by
  intro t ht g hg
  rcases ht with ⟨s, hs⟩
  rcases hs with (hs | hs)
  ·
    rw [hs]
    exact h1_conj_unip c s g hg
  ·
    rw [hs]
    exact h2_conj_unip c s g hg

theorem torusGens_inv (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, t-1 ∈ GM.torusGens c := by
  intro t ht
  rcases ht with ⟨s, hs | hs⟩
  · rw [hs]
    refine ⟨s-1, Or.inl ?_⟩
    rw [h1_inv_mk c s]
  · rw [hs]
    refine ⟨s-1, Or.inr ?_⟩
    rw [h2_inv_mk c s]

theorem conj_closure_of_gens {G : Type*} [Group G] (T : Set G) (x : G)
  (hx : ∀ g ∈ T, x * g * x-1 ∈ Subgroup.closure T) :
  ∀ u ∈ Subgroup.closure T, x * u * x-1 ∈ Subgroup.closure T := by
  intro u hu
  induction hu using Subgroup.closure_induction with
  | mem g hg =>
    exact hx g hg
  | one =>
    simp
  | mul a b ha hb iha ihb =>
    have h : x * (a * b) * x-1 = (x * a * x-1) * (x * b * x-1) := by group
    rw [h]
    exact Subgroup.mul_mem _ iha ihb
  | inv a ha iha =>
    have hxa : x * a-1 * x-1 = (x * a * x-1)-1 := by group
    rw [hxa]
    exact Subgroup.inv_mem _ iha

theorem closure_normalizes {G : Type*} [Group G] (S T : Set G)
  (hinv : ∀ t ∈ S, t-1 ∈ S)
  (hconj : ∀ t ∈ S, ∀ u ∈ Subgroup.closure T, t * u * t-1 ∈ Subgroup.closure T) :
  ∀ h ∈ Subgroup.closure S, ∀ u ∈ Subgroup.closure T, h * u * h-1 ∈ Subgroup.closure T := by
  intro h hh
  induction hh using Subgroup.closure_induction_left with
  | one =>
    intro u hu
    simp using hu
  | mul_left x hx y hy ih =>

```

```

    intro u hu
    have hcalc : (x * y) * u * (x * y)-1 = x * (y * u * y-1) * x-1 := by
      simp [mul_assoc]
    rw [hcalc]
    exact hconj x hx (y * u * y-1) (ih u hu)
| inv_mul_cancel x hx y hy ih =>
  intro u hu
  have hcalc : (x-1 * y) * u * (x-1 * y)-1 = x-1 * (y * u * y-1) * x := by
    simp [mul_assoc]
  rw [hcalc]
  have hcalc2 : x-1 * (y * u * y-1) * x = x-1 * (y * u * y-1) * (x-1)-1 := by
    simp
  rw [hcalc2]
  have hx_inv : x-1 ∈ S := hinv x hx
  exact hconj x-1 hx_inv (y * u * y-1) (ih u hu)

theorem gsimple_torus_normalizes_unipotent (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c, h * u * h-1 ∈ GM.unipSub c := by
  intro h hh u hu
  exact closure_normalizes (GM.torusGens c) (GM.unipGens c) (torusGens_inv c)
  (fun t ht => conj_closure_of_gens (GM.unipGens c) t
    (fun g hg => Subgroup.subset_closure (torusGens_conj c t ht g hg))) h hh u hu

-- From run srv_20261006_173043_GSimple_Lemma_3_5 (GSimple_Lemma_3_5), proved.

theorem conj_inv_rewrite {G : Type*} [Group G] (h u : G) :
  u * h = h * (h-1 * u * h-1-1) := by
  group

theorem gsimple_torus_unipotent_commute (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c,
  ∃ u' ∈ GM.unipSub c, u * h = h * u' := by
  intro h hh u hu
  refine ⟨h-1 * u * h-1-1,
    gsimple_torus_normalizes_unipotent c h-1 (Subgroup.inv_mem _ hh) u hu, ?_⟩
  exact conj_inv_rewrite h u

theorem hu_mul (c : ℕ → ℕ) (a b : GM.Grp c)
  (ha : ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a = h * u)
  (hb : ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, b = h * u) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a * b = h * u := by
  rcases ha with ⟨h1, hh1, u1, hu1, ha_eq⟩
  rcases hb with ⟨h2, hh2, u2, hu2, hb_eq⟩
  rcases gsimple_torus_unipotent_commute c h2 hh2 u1 hu1 with ⟨u1', hu1', hu_comm⟩
  refine ⟨h1 * h2, Subgroup.mul_mem _ hh1 hh2, u1' * u2, Subgroup.mul_mem _ hu1' hu2, ?_⟩
  calc
    a * b = (h1 * u1) * (h2 * u2) := by rw [ha_eq, hb_eq]
    _ = h1 * (u1 * h2) * u2 := by group
    _ = h1 * (h2 * u1') * u2 := by rw [hu_comm]
    _ = (h1 * h2) * (u1' * u2) := by group

theorem hu_inv (c : ℕ → ℕ) (a : GM.Grp c)
  (ha : ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a = h * u) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, a-1 = h * u := by
  rcases ha with ⟨h, hh, u, hu, rfl⟩
  have h_inv : h-1 ∈ GM.torusSub c := Subgroup.inv_mem _ hh
  have u_inv : u-1 ∈ GM.unipSub c := Subgroup.inv_mem _ hu

```

```

have h_comm := gsimple_torus_unipotent_commute c (h-1) h_inv (u-1) u_inv
rcases h_comm with ⟨u', hu', h_eq⟩
refine ⟨h-1, h_inv, u', hu', ?_⟩
calc
  (h * u)-1 = u-1 * h-1 := by group
  _ = h-1 * u' := h_eq

theorem hu_one (c : ℕ → ℕ) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, (1 : GM.Grp c) = h * u := by
have h1 : (1 : GM.Grp c) ∈ GM.torusSub c := Subgroup.one_mem _
have h2 : (1 : GM.Grp c) ∈ GM.unipSub c := Subgroup.one_mem _
refine ⟨1, h1, 1, h2, ?_⟩
simp

theorem hu_of_torus (c : ℕ → ℕ) (x : GM.Grp c) (hx : x ∈ GM.torusSub c) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, x = h * u := by
refine ⟨x, hx, 1, Subgroup.one_mem _, ?_⟩
simp

theorem hu_of_unip (c : ℕ → ℕ) (x : GM.Grp c) (hx : x ∈ GM.unipSub c) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, x = h * u := by
refine ⟨1, Subgroup.one_mem _, x, hx, ?_⟩
simp

theorem hu_gen (c : ℕ → ℕ) (g : GM.Gen c) :
  ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c,
    (PresentedGroup.of g : GM.Grp c) = h * u := by
induction g with
| H1 s =>
  refine ⟨PresentedGroup.of (GM.Gen.H1 s), Subgroup.subset_closure (by
    refine ⟨s, Or.inl rfl⟩), 1, Subgroup.one_mem _, ?_⟩
  rfl
| H2 s =>
  refine ⟨PresentedGroup.of (GM.Gen.H2 s), Subgroup.subset_closure (by
    refine ⟨s, Or.inr rfl⟩), 1, Subgroup.one_mem _, ?_⟩
  rfl
| Xm u =>
  refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.Xm u),
    Subgroup.subset_closure (Or.inl ⟨u, rfl⟩), ?_⟩
  rfl
| Ym u =>
  refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.Ym u),
    Subgroup.subset_closure (Or.inr (Or.inl ⟨u, rfl⟩)), ?_⟩
  rfl
| X l j k h u =>
  refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.X l j k h u),
    Subgroup.subset_closure (Or.inr (Or.inr (Or.inl ⟨1, j, k, h, u, rfl⟩))), ?_⟩
  rfl
| Y l j k h u =>
  refine ⟨1, Subgroup.one_mem _, PresentedGroup.of (GM.Gen.Y l j k h u),
    Subgroup.subset_closure (Or.inr (Or.inr (Or.inr ⟨1, j, k, h, u, rfl⟩))), ?_⟩
  rfl

theorem solution (c : ℕ → ℕ) :
  ∀ g : GM.Grp c, ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, g = h * u := by
intro g
let S : Set (GM.Grp c) := {x | ∃ h ∈ GM.torusSub c, ∃ u ∈ GM.unipSub c, x = h * u}
let N : Subgroup (GM.Grp c) :=
  { carrier := S

```



```

mul_mem' := fun {a b} ha hb => hu_mul c a b ha hb
one_mem' := hu_one c
inv_mem' := fun {a} ha => hu_inv c a ha }
have hN : N =  $\top$  := subgroup_top_of_gens c N (fun x => hu_gen c x)
have hg : g  $\in$  N := by
  rw [hN]
  exact Subgroup.mem_top g
exact hg

```